

SMCB_Homework04

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyana Slavchev
Görkem Kadir Solun

2026-03-16

Problem 8

a) What are the estimated match emission probabilities of the profile HMM in Figure 1?

$$E_i(a) = \frac{N_i(a) + 1}{\sum_{a' \in \mathcal{A}} (N_i(a') + 1)}$$

For each nucleotide at each position the probabilities are calculated. Example for position 1 with nucleotide A:

$$E_1(A) = \frac{4 + 1}{(4 + 1) + (0 + 1) + (0 + 1) + (0 + 1)} = \frac{5}{8} = 0.625$$

For all position and nucleotides:

	Position 1	Position 2	Position 3
A	0.625	0.125	0.111
C	0.125	0.125	0.667
G	0.125	0.625	0.111
T	0.125	0.125	0.111

b) Estimate insert emission probabilities of the profile HMM in Figure 1

Using the same formula as in task a) but for the insertion states. Example for position 1 with nucleotide A:

$$E_{\text{Insertion } 1}(A) = \frac{0 + 1}{(0 + 1) + (0 + 1) + (0 + 1) + (0 + 1)} = \frac{1}{4} = 0.25$$

For all position and nucleotides:

	Position 1	Position 2	Position 3
A	0.25	0.6	0.25
C	0.25	0.1	0.25
G	0.25	0.2	0.25
T	0.25	0.1	0.25

c) What are the estimated transition probabilities in the profile HMM in Figure 1?

$$T_i(k \rightarrow \ell) = \frac{N_i(k \rightarrow \ell) + 1}{\sum_{\ell' \in \mathcal{S}} (N_i(k \rightarrow \ell') + 1)}$$

For each state at each position the probabilities are calculated. With the initial state probabilities for position 1:

	Probabilities
Start $\rightarrow M$	0.625
Start $\rightarrow I$	0.125
Start $\rightarrow D$	0.250

Example for position 2 with the previous state M transitioning to state M:

$$T_1(M \rightarrow M) = \frac{3 + 1}{(3 + 1) + (1 + 1) + (0 + 1)} = \frac{4}{7} = 0.571$$

	T_1	T_2	T_3
$M \rightarrow M$	0.571	0.429	0.750
$M \rightarrow I$	0.143	0.429	0.125
$M \rightarrow D$	0.286	0.143	0.125
$I \rightarrow M$	0.333	0.444	0.333
$I \rightarrow I$	0.333	0.444	0.333
$I \rightarrow D$	0.333	0.111	0.333
$D \rightarrow M$	0.500	0.250	0.333
$D \rightarrow I$	0.250	0.500	0.333
$D \rightarrow D$	0.250	0.250	0.333

Problem 9

```
source("code/profileHMM.R")

library(ggplot2)
library(patchwork)
library(knitr)
```

a) Read the two alignments ‘GTP_binding_proteins.txt’ and ‘ATPases.txt’ into memory using the function `parseAlignment()` from “profileHMM.R”

```
# Read the two alignments
GTP_binding_proteins <- parseAlignment("data/GTP_binding_proteins.txt")
ATPases <- parseAlignment("data/ATPases.txt")
```

b) Use the function `learnHMM()` from “profileHMM.R” to parametrise two profile HMMs: one for each protein family (multiple alignment)

```
parGTP <- learnHMM(GTP_binding_proteins)
parATP <- learnHMM(ATPases)
```

c) For each family, identify the position(s) with the highest match and with the highest insert emission frequencies over all symbols. Plot the respective match and insert emission frequencies for the identified positions

```
# Identify which col has the highest freq
maxMatchGTP <- which(parGTP$mE == max(parGTP$mE, na.rm = TRUE),
                    arr.ind = TRUE)
maxInsertGTP <- which(parGTP$iE == max(parGTP$iE, na.rm = TRUE),
                    arr.ind = TRUE)

maxMatchATP <- which(parATP$mE == max(parATP$mE, na.rm = TRUE),
                    arr.ind = TRUE)
maxInsertATP <- which(parATP$iE == max(parATP$iE, na.rm = TRUE),
                    arr.ind = TRUE)

# Collect data in one data frame for GTP
df_GTP <- data.frame(
  Amino = rownames(parGTP$mE),
  MatchProb = parGTP$mE[, maxMatchGTP[2]],
  InsertProb = parGTP$iE[, maxInsertGTP[2]]
)

# Create plots for match and insertion
plotMatchGTP <- ggplot(df_GTP, aes(x = MatchProb, y = Amino)) +
  geom_col() +
  theme_minimal() +
  labs(title = paste("Match state at Position", maxMatchGTP[2]),
       x = "Frequency",
       y = "Amino Acid") +
  scale_y_discrete(limits=rev)

plotInsertGTP <- ggplot(df_GTP, aes(x = InsertProb, y = Amino)) +
  geom_col() +
  theme_minimal() +
  labs(title = paste("Insertion state at Position", maxInsertGTP[2]),
       x = "Frequency",
       y = "Amino Acid") +
  scale_y_discrete(limits=rev)

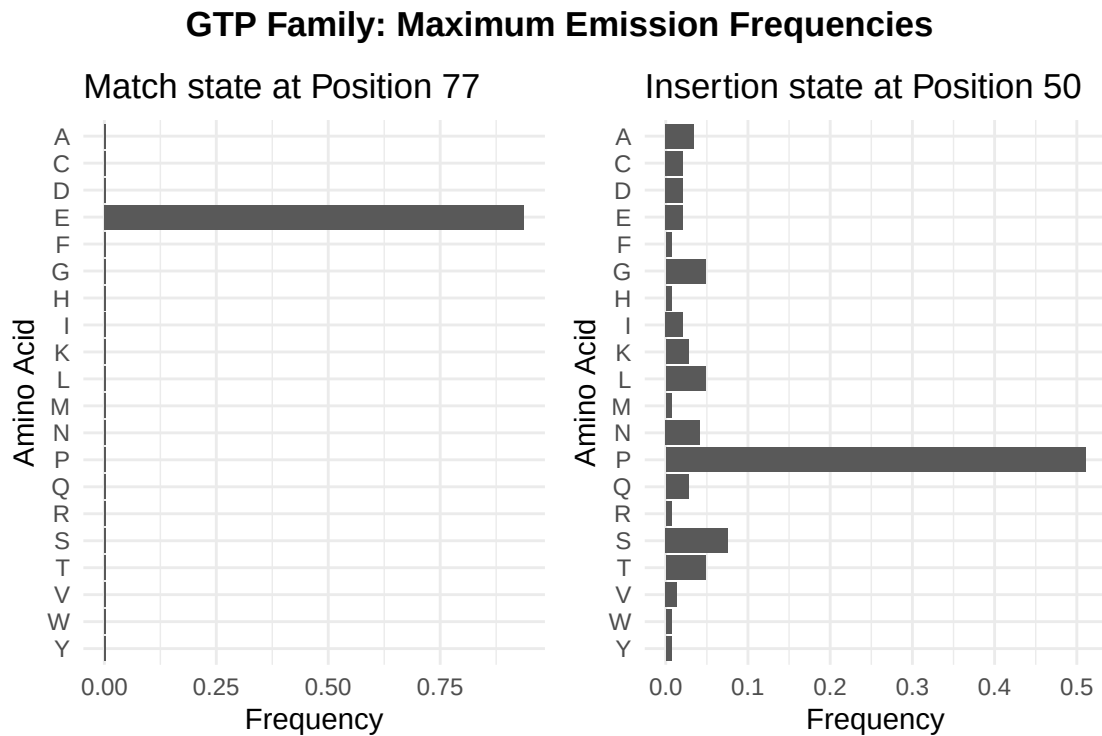
# Combine into one plot
combinedGTP <- plotMatchGTP + plotInsertGTP +
  plot_annotation(
```

```

title = "GTP Family: Maximum Emission Frequencies",
theme = theme(plot.title = element_text(face = "bold", hjust = 0.5))
)

combinedGTP

```



```

# Collect data in one data frame for ATP
df_ATP <- data.frame(
  Amino = rownames(parATP$mE),
  MatchProb = parATP$mE[, maxMatchATP[2]],
  InsertProb = parATP$iE[, maxInsertATP[2]]
)

# Create plots for match and insertion
plotMatchATP <- ggplot(df_ATP, aes(x = MatchProb, y = Amino)) +
  geom_col() +
  theme_minimal() +
  labs(title = paste("Match state at Position", maxMatchATP[2]),
       x = "Frequency",
       y = "Amino Acid") +

```

```

scale_y_discrete(limits=rev)

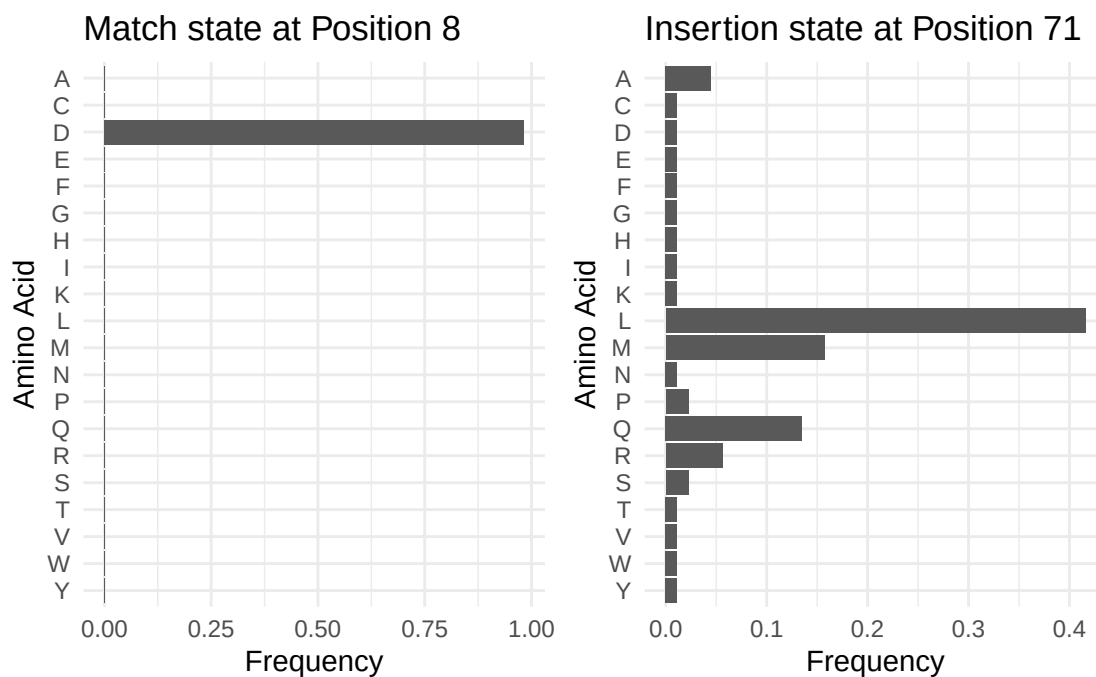
plotInsertATP <- ggplot(df_ATP, aes(x = InsertProb, y = Amino)) +
  geom_col() +
  theme_minimal() +
  labs(title = paste("Insertion state at Position", maxInsertATP[2]),
       x = "Frequency",
       y = "Amino Acid") +
  scale_y_discrete(limits=rev)

# Combine into one plot
combinedATP <- plotMatchATP + plotInsertATP +
  plot_annotation(
    title = "ATP Family: Maximum Emission Frequencies",
    theme = theme(plot.title = element_text(face = "bold", hjust = 0.5))
  )

combinedATP

```

ATP Family: Maximum Emission Frequencies



d) Complete the forward() function by filling out the recursion step.

```
#
# The Forward algorithm
#
forward <- function(HMM, seq) {
  L <- HMM$L
  mE <- HMM$mE
  iE <- HMM$iE
  T <- HMM$T
  len <- length(seq)

  # Random model: same emission probability for all letters in the alphabet
  qr <- 1/nrow(mE)
  # Initialize score matrices for match, insert and delete
  Fm<- Fi<- Fd<- matrix(-Inf, nrow=len+1, ncol=L+1)
  # First column for the begin state (M0) (indices get shifted by 1)
  Fm[1,1]<-0

  # We then allow transitions to I0 and D1 (start with insertion or deletion)
  for(i in 2:len+1){
    Fi[i,1] <- log(iE[seq[i-1],1]/qr) +
      log(T['MI',1]*exp(Fm[i-1,1]) +
          T['II',1]*exp(Fi[i-1,1]))
  }

  # i is the pos in the sequence, j is the profile state
  for(i in 2:(len+1)) {
    for(j in 2:(L+1)) {
      ##### your code here to compute
      #Fm[i, j], Fi[i,j], and Fd[i,j]

      # Calculate the score for the match state
      Fm[i,j] = log(mE[seq[i-1],j]/qr) + log(
        T["MM", j-1] * exp(Fm[i-1, j-1])
        + T["IM", j-1] * exp(Fi[i-1, j-1])
        + T["DM", j-1] * exp(Fd[i-1, j-1])
      )

      # Calculate the score for the insertion state
      Fi[i,j] = log(iE[seq[i-1],j]/qr) + log(
        T["MI", j] * exp(Fm[i-1, j])
```

```

    + T["II", j] * exp(Fi[i-1, j])
    + T["DI", j] * exp(Fd[i-1, j])
  )

  # Calculate the score for the deletion state
  Fd[i,j] = log(T["MD", j-1] * exp(Fm[i, j-1])
               + T["ID", j-1] * exp(Fi[i, j-1])
               + T["DD", j-1] * exp(Fd[i, j-1]))

  ##### your code ends
}
}

# Termination step: Fe = Fm[len,L+2] = log(P(x|M)/P(x|R))
Fe <- log(T["MM",L+1]*exp(Fm[len+1,L+1]) +
          T["IM",L+1]*exp(Fi[len+1,L+1]) +
          T["DM",L+1]*exp(Fd[len+1,L+1]))
return(Fe)
}

```

e) The file `Unclassified_proteins.txt` contains 31 protein sequences from unknown families. Load the protein sequences into a list using the `parseProteins()` function from in “`profileHMM.R`”.

```

proteinList <- parseProteins("data/Unclassified_proteins.txt")

```

f) For each unclassified protein $x^{(i)}$ in the list, apply the forward algorithm for both models M_1 and M_2 to obtain the log odds ratio. Plot the values $q(x^{(i)})$. Which proteins in the list belong to which family? Can you clearly decide for each protein?

```

# Iterate over the protein sequences and calculate the log odds ratio
logOdds <- c(rep(0, length(proteinList)))

for (i in seq_along(proteinList)) {
  logOddsGTP <- forward(parGTP, proteinList[[i]])
  logOddsATP <- forward(parATP, proteinList[[i]])
}

```

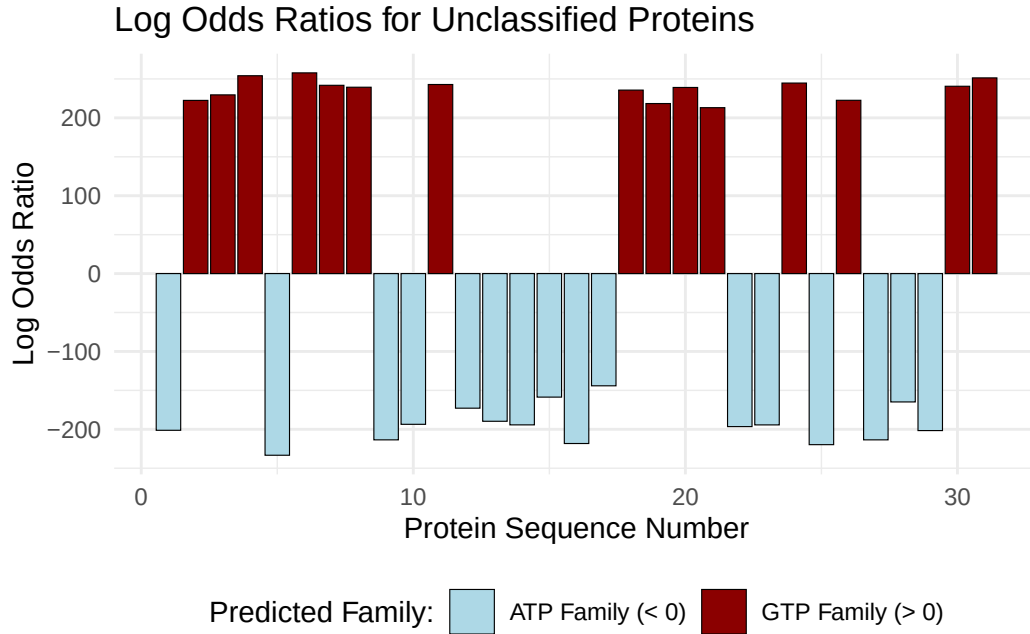
```

# Subtract to get the ratio since log
logOdds[i] <- logOddsGTP - logOddsATP
}

# Plot the ratios
df_ratio <- data.frame(
  seqNum = seq(1:length(proteinList)),
  ratio = logOdds
)

ggplot(df_ratio, aes(x = seqNum, y = ratio, fill = ratio > 0)) +
  geom_col(color = "black", linewidth = 0.2) +
  theme_minimal() +
  scale_fill_manual(
    name = "Predicted Family:",
    values = c("TRUE" = "darkred", "FALSE" = "lightblue"),
    labels = c("TRUE" = "GTP Family (> 0)", "FALSE" = "ATP Family (< 0)")
  ) +
  labs(
    title = "Log Odds Ratios for Unclassified Proteins",
    x = "Protein Sequence Number",
    y = "Log Odds Ratio"
  ) +
  theme(legend.position = "bottom")

```



```
# Create a table for each family predicition
df_ratio$Family <- ifelse(df_ratio$ratio > 0, "GTP", "ATP")
df_ratio$Score <- round(df_ratio$ratio, 3)

gtp_proteins <- df_ratio[df_ratio$Family == "GTP", c("seqNum", "Score")]
atp_proteins <- df_ratio[df_ratio$Family == "ATP", c("seqNum", "Score")]

# GTP Table
kable(gtp_proteins,
      col.names = c("Sequence number", "Log-Odds score"),
      align = "c",
      caption = "Proteins classified as GTP family, positive score",
      row.names = FALSE)
```

Table 5: Proteins classified as GTP family, positive score

Sequence number	Log-Odds score
2	222.448
3	229.542
4	254.115
6	257.799
7	241.813

Sequence number	Log-Odds score
8	239.352
11	242.864
18	235.651
19	218.396
20	239.051
21	213.103
24	244.696
26	222.585
30	240.588
31	251.338

```
# ATP Table
kable(atp_proteins,
      col.names = c("Sequence number", "Log-Odds score"),
      align = "c",
      caption = "Proteins classified as ATP family, negative score",
      row.names = FALSE)
```

Table 6: Proteins classified as ATP family, negative score

Sequence number	Log-Odds score
1	-201.276
5	-233.324
9	-213.540
10	-193.592
12	-172.862
13	-189.699
14	-194.320
15	-158.669
16	-218.264
17	-144.132
22	-196.524
23	-194.348
25	-219.736
27	-213.547
28	-164.847
29	-201.635

The prediction for each protein is visible in the tables above. The decision of which protein belongs to which family is clear in this case, since the log-odds ratio closest to zero across both

families (GTP: 213.103, ATP: -144.132) remains far from 0, indicating confident classification for all proteins.